



Diraya

Saudi Technology Research Hub

An automated Azure data pipeline and reference website for technology research
across six Saudi universities

SUBMITTED BY

Rana Ayman Almohethef

Rana Saad AlHasaniah

Aryam Saad Alotaibi

September 2026

Table of Contents

1. Executive Summary	3
2. Problem Statement.....	3
2.1 The Current Reality	3
2.2 Pain Points	3
2.3 The Cost of Inaction	3
3. Proposed Solution	3
3.1 What Is It?	3
3.2 Core Capabilities.....	4
3.3 How It Works (The Drone Example).....	4
4. Target Audience.....	4
5. Technical Approach	4
5.1 Technology Stack and Pipeline Architecture	4
6. Solution Architecture	5
6.1 The Nine Stages.....	6
6.2 Orchestration and Monitoring.....	7
6.3 Data Governance	7
6.4 Why It Is Built This Way	8
7. Development Timeline	8
8. Evaluation Plan.....	8
8.1 Pre-Publishing Verification.....	9
8.2 Post-Publishing Check.....	9
8.3 Task-Based User Evaluation	9
9. Risks and Mitigation.....	9
10. Scope and Boundaries.....	10
11. Expected Outcomes	10
12. Conclusion.....	10

1. Executive Summary

Technology research from Saudi universities is scattered across journals, institutional repositories, and databases reached through different APIs. Researchers and postgraduate students must search several sources by hand to find relevant studies, compare methodologies, and build on previous findings. This fragmentation limits the visibility of Saudi research and makes it harder to identify research gaps, potential collaborators, and solutions that could benefit other sectors.

Diraya brings this research together through an automated data pipeline and a searchable website. By collecting, standardizing, and regularly updating research records, the platform helps users discover studies, explore methodologies, and build on existing work. To date, the pipeline has filtered and consolidated 5,344 technology-related research papers out of 22,960 validated records covering 2023–2026. The result is a central reference for Saudi technology research that supports collaboration and makes local research easier to reach for researchers and decision-makers.

2. Problem Statement

2.1 The Current Reality

During our undergraduate projects we had to review previous studies, compare methodologies, and judge which approaches would support our intended outcomes. That meant searching for several scattered sources by hand and reviewing findings across different platforms.

2.2 Pain Points

- **Fragmented information:** some records contained only abstracts or brief methodological summaries, which made it difficult to understand how a study had been conducted.
- **Missing metadata:** when a record lacked a publication link or clear author and university information, locating the original paper or identifying the researcher required extra searching.
- **Lost opportunities for collaboration:** without clear institutional affiliation and contact points, we could not find fuller explanations or reach authors for clarification.

2.3 The Cost of Inaction

Without a central repository, the effort needed to separate Saudi research from broader international results stays unnecessarily high. That slows literature reviews, limits academic communication, and reduces the visibility of local technological work.

3. Proposed Solution

3.1 What Is It?

Diraya is a reference website for Saudi technology research, supported by an automated cloud data pipeline. The pipeline collects research metadata (titles, authors, university affiliation, and publication

links where available) across six major Saudi universities: KSU, KAU, KKU, PSAU, KFUPM, and KAUST. It also uses external sources such as OpenAlex and Crossref for validation and enrichment.

3.2 Core Capabilities

Data processing follows a strict, automated flow:

- **Collect:** extraction of research records from university open data portals, APIs, and repositories.
- **Clean:** normalizing nested formats (XML, JSON) and standardizing university names and author fields.
- **Filter:** applying a technology filter built on 33 whole-word keywords to isolate relevant research. The keywords are matched against the title and the abstract for every source except KSU, whose records carry author keywords instead of an abstract, so KSU is matched on the title and those keywords.
- **Enrich and validate:** cross-referencing missing DOIs and validating fields through external APIs.
- **Store and serve:** publishing the final dataset to PostgreSQL and serving it through a website where users can search for studies and follow links to the original sources.

3.3 How It Works (The Drone Example)

Bringing these records together lets users target a specific technology quickly. A researcher in Riyadh interested in recent drone research can use the website to find the studies that belong to Saudi universities, review the publication details, identify the authors, and check their affiliations. This narrows the search effort and separates Saudi research from broader international results.

4. Target Audience

- **Primary users:** undergraduate and postgraduate students reviewing literature and selecting research methodologies.
- **Secondary users:** researchers and industry professionals who want to understand work carried out at Saudi universities or identify collaborators in a related field.

5. Technical Approach

The project uses a modern data engineering architecture hosted on Microsoft Azure, built for automation, repeatable runs, and data quality.

5.1 Technology Stack and Pipeline Architecture

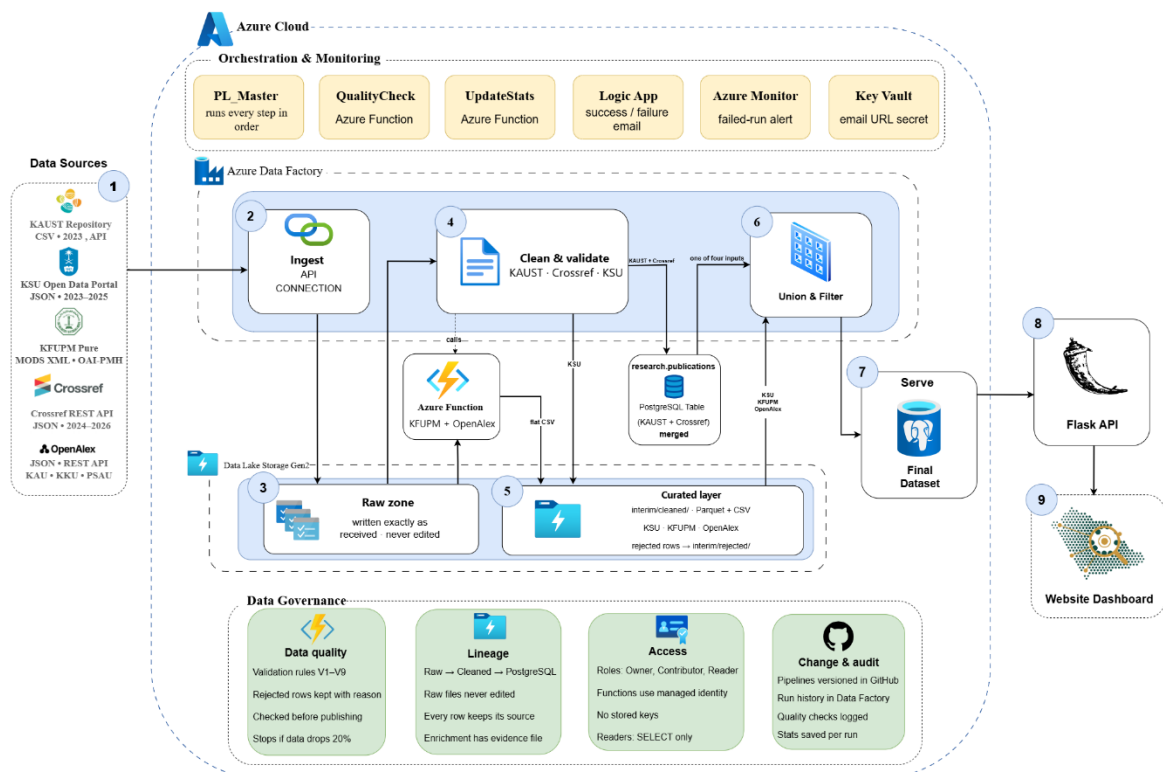
- **Data ingestion (Azure Data Factory):** one master pipeline (PL_Master) runs child pipelines in order and extracts raw research data from university APIs, DSpace and Pure repositories, and external APIs (Crossref, OpenAlex).

- **Data storage (Azure Data Lake Storage Gen2):** raw files (JSON, XML, CSV) are kept untouched in a raw zone, and cleaned interim data is written to a curated zone.
- **Data processing (Azure Functions and ADF Data Flows):** nested formats such as KFUPM's MODS XML and OpenAlex's inverted-index abstracts are managed by a Python 3.11 Azure Function App, while the remaining transformations run as ADF mapping data flows.
- **Database (Azure Database for PostgreSQL):** the validated, technology-filtered records are published to a PostgreSQL flexible server as one final dataset of 15 standardized fields.
- **Serving layer (Flask API and website dashboard):** the website reads the data through a small Flask API (/api/papers/all, /api/stats, /api/meta), so every number on the page comes from the database rather than from typed-in values.
- **Orchestration and security:** Azure Logic Apps send a success or failure email after every run, Azure Monitor raises an alert whenever a scheduled run fails, and Azure Key Vault holds the notification URL so that no secret is stored inside the pipeline definitions.

6. Solution Architecture

The diagram below follows one research record from the moment it is collected to the moment it appears on the website. The numbered badges mark the nine stages of that journey. Everything inside the dashed boundary runs in Azure: the boxes at the top orchestrate and watch each run, and the boxes at the bottom describe the rules that apply across the whole pipeline.

Figure 1 — Solution architecture



Sources (1) → ingestion (2) → raw zone (3) → cleaning and validation (4) → curated zone (5) → union and technology filter (6) → PostgreSQL (7) → Flask API (8) → website dashboard (9).

6.1 The Nine Stages

1. Data Sources

Five public sources feed the platform: the KAUST repository (CSV and API, 2023), the KSU open data portal (JSON, 2023–2025), KFUPM Pure (MODS XML over OAI-PMH), the Crossref REST API (JSON, 2024–2026), and OpenAlex (REST API) for KAU, KKU, and PSAU. Most data are collected automatically from public APIs and university websites. KAUST’s 2023 records are read from a previously downloaded CSV file.

2. Ingestion — Azure Data Factory

Copy activities and API connections pull each source on its own child pipeline. Sources with nested formats (KFUPM XML, OpenAlex JSON) are handed to the Azure Function App, which returns a flat structure Data Factory can write.

3. Raw Zone — ADLS Gen2

Every response is written to the raw container exactly as it arrived. Raw files are not modified during cleaning and remain available for reprocessing until replaced by a subsequent ingestion run.

4. Cleaning and Validation

Data flows and Python functions standardize university names, author fields, titles, DOIs, URLs, and dates. Validation rules V1–V9 run on each source; rejected rows are kept together with the reason they failed instead of being dropped silently. These per-source rules are separate from the eleven SQL gates described in section 8.1, which run later against the combined candidate table.

5. Curated Zone — ADLS Gen2

The cleaned output of every source is written back to the curated container in a consistent schema, ready to be combined. This is the boundary between per-source logic and the shared pipeline.

6. Union and Technology Filter

The cleaned outputs of the six universities (produced by the five source routes) are combined into one table, duplicates are removed by `research_id`, and a filter of 33 technology keywords keeps only technology-related research. The result is the final candidate table.

7. Serve — PostgreSQL

After the quality gates pass, the candidate table is published to `research.final_dataset` inside one transaction: the table is replaced only if every row is inserted. As of the run of 26 September 2026 the published dataset holds 5,344 technology papers in 15 standardized fields.

8. Flask API

A small Flask service is the only door between the website and the database. It exposes `/api/papers/all`, `/api/stats`, and `/api/meta`, so the browser never talks to PostgreSQL directly and no database credentials leave the server.

9. Website Dashboard

The dashboard reads those endpoints and renders the analytics, the dataset explorer, and the pipeline numbers. Every figure on the page (totals, per-university counts, the last update date) comes from the database, so the site updates itself whenever the pipeline runs.

6.2 Orchestration and Monitoring

These components do not transform data. They decide when the pipeline runs, and they make a failed run visible instead of silent.

Component	What it does
PL_Master	One master pipeline that runs every child pipeline in a fixed order, so a step never starts before the step it depends on has finished.
QualityCheck (Azure Function)	Re-checks the published table after the run with 8 independent checks and writes the result to research.quality_checks. A failed check fails the pipeline run.
UpdateStats (Azure Function)	Records how many rows each source produced at each stage, which is what the dashboard shows as the per-university comparison.
Logic App	Sends a success or failure email at the end of every run, including the run ID and the row count.
Azure Monitor	Raises an alert when a triggered run fails, so the team is notified even if nobody is watching the pipeline.
Azure Key Vault	Holds the notification URL as a secret. The pipeline reads it at run time through its managed identity, so the URL is never written inside the pipeline definitions in GitHub.

6.3 Data Governance

These practices apply from the raw files to the final dataset.

Area	Practice
Data quality	Validation rules V1–V9 on every source · rejected rows are kept with their reason · the candidate table is checked before publishing · the run stops if any university loses more than 20% of its rows.
Lineage	One direction only: raw → cleaned → PostgreSQL. Raw files are never edited, and every row keeps its source and its research_id. Manual enrichment is documented in its own evidence file.

Area	Practice
Access	Azure roles split into Owner, Contributor, and Reader. Functions reach storage through managed identity rather than stored keys, and read-only users get SELECT only.
Change and audit	Pipelines are versioned in GitHub, run history is kept in Data Factory Monitor, and the quality_checks and source_stats tables record the outcome of every run.

6.4 Why It Is Built This Way

- **Separate raw and curated zones:** keeping the original response untouched means a cleaning bug can be fixed and replayed without going back to the source, which may no longer return the same data.
- **One master pipeline:** a single-entry point makes the order explicit and gives one place to read the run history, instead of six independent schedules.
- **Checks before and after publishing:** the pre-publish gate protects the live table, and the post-publish check proves what was published.
- **An API between the site and the database:** the website can be shared without exposing the database, and the same endpoints can later serve other applications.

7. Development Timeline

- **Week 1 — Requirements and architecture:** define the data sources across the six universities, finalize the Azure architecture, and map the target metadata schema.
- **Week 2 — Ingestion and storage:** build the ADF pipelines that land raw data in ADLS Gen2 from KSU, KAUST, KFUPM, OpenAlex, and Crossref.
- **Week 3 — Processing:** develop the Python Azure Functions and the ADF data flows that clean the formats, standardize the text, and apply the technology filter.
- **Week 4 — Validation and database:** implement the quality gates and load the final candidate table into PostgreSQL.
- **Week 5 — Serving and dashboard:** connect PostgreSQL to the Flask API, build the dashboard, and configure the Logic App notifications.

8. Evaluation Plan

The project applies programmatic quality gates both before and after the final dataset is published.

Before those gates, the pipeline code itself is covered by 129 unit tests held in the source repository: 52 for schema validation, 31 for cleaning and normalization, 22 for the technology filter and source combination, 16 for OpenAlex processing, and 8 for file writing. The suite runs automatically through GitHub Actions on every push, so a change that breaks a cleaning or validation rule is caught before it is

deployed. Several tests encode judgement rather than mechanics: one confirms that “computer vision syndrome” is never counted as computer vision research, another confirms that the manual enrichment file cannot overwrite a DOI that already exists and disagrees with it, and others confirm that partial dates are never completed and that a publication date is never borrowed from OpenAlex.

8.1 Pre-Publishing Verification

Before anything is published, 11 SQL validations run against the candidate table inside the pipeline. They confirm that the table is not empty, that the seven required fields (research_id, title, authors, DOI, URL, source, publication year) are present, that DOIs are lowercase and correctly formatted, that URLs are valid, that publication years fall inside the expected range, that research_id values are unique, and that all six universities are represented. The last of the eleven is a safeguard that stops the run when any single university loses more than 20% of its rows compared with the previous successful run, so a silent data loss cannot reach the published table.

Publishing itself runs inside a single database transaction: the final table is replaced only if every row is inserted successfully, so the website never shows a half-written dataset.

8.2 Post-Publishing Check

After publishing, an Azure Function (quality_check) re-evaluates the published table with 8 independent checks, writes the result to a dedicated quality_checks table, and compares the new total with the last successful run. If any check fails the function returns an error and the pipeline run is marked as failed, which triggers the failure email.

8.3 Filter Accuracy and User Evaluation

The technology filter itself was reviewed by hand on a sample of about 60 papers drawn from the published dataset and from the rows the filter rejected. On that sample the filter kept the right paper in roughly 93% of the cases it selected, and it recovered roughly 68% of the technology papers present, giving an overall agreement of about 88% with the manual reading. These are indicative figures from a small sample, not a formal measurement; a labelled evaluation over 200 randomly drawn papers is the planned next step, and the main known gap is recall, because a paper that uses none of the 33 keywords is not kept.

We also measure how quickly end users can query a specific topic, such as drone research, and reach the original source from the dashboard.

9. Risks and Mitigation

- **API rate limits and IP blocks:** external sources such as KAUST block cloud IP ranges. Mitigation: stage that data manually where necessary and use scheduled batch processing for the accessible APIs.

- **Nested and complex data formats:** ADF data flows struggle with complex XML and JSON. Mitigation: use Python-based Azure Functions written specifically to parse MODS XML and inverted-index abstracts.
- **Records that need manual correction:** a small number of KSU records were missing or malformed metadata. Mitigation: corrections are kept in a separate enrichment file with the expected original title recorded for each record, so every manual change is documented, repeatable, and applied only to the intended row.

10. Scope and Boundaries

- **In scope:** technology-related research from KSU, KAU, KKU, PSAU, KFUPM, and KAUST (2023–2026), automated metadata extraction, standardization, quality checks, and dashboard visualization.
- **Out of scope:** hosting the full text of research papers. Publication links support further investigation, but access to the full papers remains subject to the source's availability and access conditions.

The record counts shown in the dashboard reflect what each source publishes and what the technology filter keeps. They are not ranking universities.

11. Expected Outcomes

1. **Expanded visibility:** Saudi technology research becomes easier to discover across all six universities.
2. **Efficient research:** students and professionals spend less time searching for literature and methodologies.
3. **Academic collaboration:** easier access to author and institutional information is a first step toward cross-university contact.
4. **Future partnerships:** working with the universities themselves would improve research coverage, metadata accuracy, and the visibility of their publications.

12. Conclusion

Our own experience identified the need for a central Saudi research reference. Using an Azure data pipeline orchestrated through Azure Data Factory, processed with Python Functions, and served through PostgreSQL and Flask, we turned a fragmented landscape into a standardized, searchable platform. The project answers an immediate need for students and researchers, and it establishes a foundation for wider academic collaboration and technological development in Saudi Arabia.